



BSc (Hons) in **Computer Science**
SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 02

Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF wall_ahead AND left_is_visited THEN turn_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A table-driven agent stores an explicit action for every possible percept sequence. The number of sequences is $|P|^T$, where P is the set of distinct percepts and T is the number of steps in the agent's lifetime. This grows exponentially with lifetime. When lifetime grows, the number of sequences also grows. In chess there are they are 10^{40} possible board states at a time. That's a huge number and there are no any way to store that type of large number.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules"

discussed in the lecture.

In SimpleReflexAgent.sense_and_act: if percept['food_here']: return 'Eat', if percept['wall_ahead']: return 'TurnLeft', and the default return 'Forward'. Each is a literal "IF condition THEN action" mapping from the current percept straight to an action, with no state in between. This is exactly the condition-action rule.

3. (Analyze) Your SimpleReflexAgent likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

A simple reflex agent is a pure function of the current percept: identical percept - identical action, always. Under partial observability, many different world states produce the same percept - standing at the bottom-left corner and standing at the bottom-right corner both just report wall_ahead: True. Without percept history, the agent cannot tell those situations apart, so it responds to both identically. Because of

4. (Evaluate) In Step 1.3, you added an internal state to your ModelBasedAgent . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The Transition Model - how the world changes when the agent acts is the state-update block at the top of sense_and_act: a Forward shifts believed (x, y) by the facing vector and a TurnLeft/TurnRight rotates believed dir by $\pm 1 \bmod 4$. That encoding of "this action moves/turns me like so" is the agent's model of the world's dynamics. The Sensor Model - how percepts relate to the true state is how it interprets wall_ahead and food_here: it only commits the believed Forward move if the previous percept had wall_ahead: False and it uses food_here to decide Eat. Combining the two - predict with the transition model, correct/confirm with the sensor model - is what lets it maintain a visited set and thereby recognise and escape loops that the reflex agent cannot.

Finalize and Lock Lab 02 Documentation

